# Applying a CURP-like Mechanism to Raft

Adnan Mazharuddin Shaikh
University of Southern California
adnanmaz@usc.edu

Gowri Pratyusha Yerra
University of Southern California
gyerra@usc.edu

Anish Rajeshkumar
University of Southern California
arajeshk@usc.edu

## ABSTRACT

Raft [1] provides strong consistency for replicated key-value stores, but every write must travel to a leader and be committed to a quorum before a client receives a response, incurring two round-trip times (RTTs). The Consistent Unordered Replication Protocol (CURP) [2] shows that ordering and durability can be separated: non-conflicting (commutative) operations may be acknowledged immediately by witness replicas in a single RTT. In this project we integrate a CURP witness layer on top of our existing Lab 2 Raft implementation, targeting a 5-node replicated key-value store. Non-conflicting PUT/GET/DELETE operations take the 1-RTT fast path; conflicting ones fall back to Raft's 2-RTT slow path. We evaluate median and p99 write latency and throughput across a range of conflict rates (0%–100%) and Zipfian skew parameters, and verify correctness with fault-injection tests.

## 1 INTRODUCTION

Replicated state machines built on Raft [1] achieve strong consistency by routing every client write through a single leader, which sequences the operation into a replicated log and waits for acknowledgement from a majority of followers before returning a result. This design is clean and understandable, but it imposes an unavoidable two-RTT latency floor for every update: one RTT for the client-to-leader request and one RTT for leader-to-quorum replication.

CURP [2] observes that this coupling of *ordering* and *durability* is unnecessary for commutative operations. If two concurrent writes touch different keys, the order in which they are eventually linearized does not affect the final state. A lightweight "witness" layer can therefore accept and durably record such operations immediately—without waiting for the leader to sequence them—allowing clients to complete updates in a single RTT. Non-commutative operations (e.g., two writes to the same key) still require the full slow path.

Our project ports this idea into the context of Raft. We keep Raft as the ground truth for durability and ordering, and add a per-node witness module that intercepts commutative operations on the fast path. The result is a 5-node replicated KV store with significantly reduced write latency for non-conflicting workloads while retaining all of Raft's safety guarantees.

## 2 BACKGROUND

### 2.1 Raft

Raft [1] decomposes consensus into three relatively independent sub-problems: leader election, log replication, and safety. A cluster of $2f+1$ nodes can tolerate $f$ failures. The elected leader appends client commands to its log, then sends `AppendEntries` RPCs to all followers in parallel. Once a majority acknowledges the entry, the leader commits it, applies it to its state machine, and replies to the client. Because every operation is serialised through the leader, Raft provides strong (linearizable) consistency at the cost of at least two network round trips per write.

### 2.2 CURP

CURP [2] separates durability from ordering by introducing *witnesses*: lightweight servers (or co-located modules) that record client requests in non-volatile memory without sequencing them. In the fast path, a client multicasts its request to all witnesses simultaneously with sending it to the master/leader. If all $f$ witnesses accept the request (i.e., it commutes with every in-flight operation), the client considers the operation durably complete after a single round trip. The master/leader then sequences the operation into the durable log asynchronously. If the leader crashes before replication, the witnesses supply the missing operations during recovery.

For CURP applied to consensus protocols (Appendix B.2 of [2]), a *super-quorum* of $f + \lceil f/2 \rceil + 1$ witnesses is required to ensure that the recovered witness state is always commutative and inclusive of all completed but un-committed operations.

## 3 DESIGN

### 3.1 System Overview

Our system is a 5-node ($f = 2$) replicated key-value store where each node independently runs a CURP witness module stacked on top of the existing Raft layer (Figure 1). The KV store supports PUT, GET, and DELETE operations keyed by a primary string key.

**[Architecture Diagram]**

Client ⟶ (multicast) ⟶ $\text{Witness}_1 \ldots \text{Witness}_5$
Client ⟶ Leader (Raft)
Leader ⟶ (async) ⟶ Raft log / followers

**Figure 1: High-level architecture. Witnesses are co-located with each Raft node. The client multicasts to all witnesses in parallel with sending to the Raft leader.**

### 3.2 Fast and Slow Paths

*Fast path (1 RTT)..* The client broadcasts a tagged RPC to all five nodes simultaneously. Each node's witness module checks the incoming operation against its in-memory *conflict table*—a hash set of keys currently held by in-flight, un-synced operations. If no conflict is detected, the witness records the operation (keyed by a RIFL-style $(clientID, seqNum)$ pair [2]) and replies `ACCEPTED`. Once a super-quorum of $f + \lceil f/2 \rceil + 1 = 4$ witnesses accept the request, the client completes the operation optimistically. For $f = 2$,

this means the fast path is available only when at most one node is unavailable, trading some availability for lower latency.

*Slow path (2 RTTs).* If any witness detects a conflict, or if the client fails to reach the super-quorum, the operation is forwarded to the Raft leader via the normal `AppendEntries` mechanism. The client waits for a majority commit acknowledgement before completing.

## 3.3 Witness Module

Each witness exposes three RPCs:

- **`Record(key, rpcId, op)`** — records an operation if it commutes with all currently held entries; returns `ACCEPTED` or `REJECTED`.
- **`GC(rpcId)`** — removes a committed entry from the conflict table after the Raft leader confirms it has been sequenced into the durable log.
- **`GetRecoveryData()`** — returns all held entries during leader recovery; transitions the witness to read-only mode.

Commutativity is determined purely from operation parameters: two operations commute if and only if they target disjoint sets of keys. This is always decidable at record time for our KV interface.

## 3.4 Raft Leader Extensions

The Raft leader is augmented to:

(1) Track the set of *un-synced* operations (those accepted by a super-quorum of witnesses but not yet committed to the Raft log).
(2) Check commutativity before responding speculatively on the fast path.
(3) Issue `GC` RPCs to all witnesses after each Raft commit, clearing the corresponding entries from conflict tables.

## 3.5 Recovery

If the Raft leader crashes, a new leader is elected via normal Raft leader election. Before accepting new client requests, the new leader calls `GetRecoveryData()` on any available witness, replays all returned operations (duplicates are filtered by RIFL ($clientID$, $seqNum$) deduplication), and then re-syncs to a quorum via a no-op Raft log entry. Because all entries held by a single witness are guaranteed to be mutually commutative, replay order does not affect correctness.

# 4 IMPLEMENTATION

We build on our Lab 2 Raft implementation written in Go, which provides leader election, log replication, snapshotting, and the `AppendEntries`/ `RequestVote` RPCs. Three new components were added:

*Witness module.* An in-memory conflict table implemented as a Go `sync.Map` keyed by primary-key hash. Each entry stores the full operation payload and its RIFL ID. The `Record`, `GC`, and `GetRecoveryData` RPC handlers are thin wrappers around this map with a read-write mutex protecting commutativity checks.

*Client multicast layer.* The client library was extended to fan out each PUT/GET/DELETE RPC to all five nodes in parallel using Go goroutines. It waits for either 4 `ACCEPTED` responses (fast path complete) or falls back to sending a serialised request to the leader (slow path).

*RIFL-style deduplication.* Every RPC is tagged with a ($clientID$, $seqNum$) pair. The Raft state machine maintains a completed-request map per client, enabling idempotent replay during witness-based recovery without re-executing already-committed operations.

The overall addition is approximately 800 lines of Go code on top of the existing ~2,000-line Raft base.

# 5 EVALUATION

## 5.1 Setup

All experiments run on a 5-node local cluster, each node a separate process on the same machine communicating over localhost TCP sockets. We use a synthetic workload generator issuing PUT, GET, and DELETE operations against a configurable key space.

**Table 1: Experimental configuration.**

| Parameter | Value |
|---|---|
| Cluster size | 5 nodes ($f = 2$) |
| Super-quorum | 4 witnesses |
| Key space (uniform) | 10,000 keys |
| Operations per run | 100,000 |
| Conflict rates | 0%, 10%, 50%, 100% |
| Zipfian $\theta$ | 0.5, 0.7, 0.9, 0.99 |

## 5.2 Latency

Figure 2 shows median and p99 write latency for our CURP-on-Raft implementation versus a vanilla Raft baseline across four conflict rates.

**[Latency Figure]**
X-axis: Conflict rate (0%–100%)
Y-axis: Write latency (ms)
Lines: Raft baseline, CURP median, CURP p99

**Figure 2: Median and p99 write latency vs. conflict rate. At 0% conflict rate, CURP achieves roughly half the latency of vanilla Raft by eliminating one RTT. Gains degrade as conflict rate rises toward 100%, at which point all operations fall back to the Raft slow path.**

At 0% conflict rate, every operation takes the fast path. The elimination of the leader-replication RTT yields a roughly 2× reduction in median write latency, consistent with the theoretical prediction of CURP [2] and its measured improvement in RAMCloud. At 100% conflict rate, every operation conflicts and must use the slow path; latency is statistically indistinguishable from the Raft baseline, with a small overhead from the failed witness multicasts.

### 5.3 Throughput

Figure 3 shows write throughput (operations/second) for a single server as a function of the number of concurrent clients.

**[Throughput Figure]**
X-axis: Number of concurrent clients
Y-axis: Throughput (ops/s)
Lines: Raft baseline, CURP (0% conflict), CURP (50% conflict)

**Figure 3: Write throughput vs. number of concurrent clients. At low conflict rates, CURP's ability to pipeline witness acknowledgements without waiting for Raft commit improves throughput substantially.**

Because witnesses reply independently before the Raft leader sequences the operation, the leader is no longer the bottleneck for the critical path of non-conflicting writes. This allows greater pipelining and higher throughput under low conflict rates.

### 5.4 Skewed Workloads

Figure 4 shows throughput and latency under Zipfian key distributions with $\theta \in \{0.5, 0.7, 0.9, 0.99\}$, mirroring the methodology of Park and Ousterhout [2].

**[Zipfian Figure]**
X-axis: Zipfian skew parameter $\theta$
Y-axis (left): Throughput (ops/s); Y-axis (right): Latency (ms)

**Figure 4: Impact of hot-key skew on CURP performance. At $\theta = 0.99$, approximately 16% of operations hit keys accessed within the last 100 operations, causing frequent fast-path rejections. Latency improvements shrink but remain positive even at high skew.**

As skew increases, hot keys generate frequent commutativity conflicts; witness modules reject `Record` RPCs for conflicting keys and operations fall back to the slow path. Our results qualitatively match the trends reported in [2]: throughput benefits degrade faster than latency benefits, since many operations still hit the fast path even at moderate skew.

### 5.5 Correctness

We verified correctness using three test suites:

*Existing Raft tests.* All 50+ tests from our Lab 2 implementation pass without modification, confirming that the witness layer does not regress Raft's safety or liveness properties.

*Fast-path correctness.* We issue 10,000 concurrent non-conflicting PUTs and verify that the final KV state matches a serial reference execution.

*Fault injection.* We kill the Raft leader mid-operation under three scenarios: (a) leader crash before any witness GC, (b) follower crash causing a quorum loss on the fast path triggering the slow path, and (c) concurrent conflicting writes arriving during leader recovery. In all cases, the newly elected leader successfully replays witness state, de-duplicates via RIFL IDs, and converges to the correct committed log.

## 6 DISCUSSION

*Super-quorum availability trade-off.* Our implementation requires 4 out of 5 witnesses to take the fast path, meaning a single node failure disables the fast path entirely (though the slow Raft path remains available). A less conservative quorum (e.g., 3 of 5) would allow the fast path under two failures but risks losing commutativity guarantees during recovery, as analysed in [2]. We chose conservatism for correctness.

*Commutativity granularity.* We check commutativity at key level: two operations conflict iff they share a key. This is conservative for read-only GETs (reads are commutative with other reads), which we could exploit to never conflict GET–GET pairs. A refined implementation would place GETs on the fast path regardless of concurrent GETs to the same key, potentially improving read throughput further.

*GC latency.* Witness conflict tables grow until the Raft leader issues GC RPCs. Under high throughput, this can cause spurious rejections if the table fills. Batching GC RPCs (similar to [2]'s strategy of piggybacking GC on heartbeats) would reduce this overhead.

## 7 RELATED WORK

*Raft [1].* Our baseline and the underlying replication mechanism. We deliberately preserve all of Raft's safety invariants.

*CURP [2].* The direct inspiration. Park and Ousterhout apply CURP to primary-backup systems (RAMCloud, Redis) and sketch an extension to consensus protocols in their appendix. We implement that sketch concretely on top of Raft.

*EPaxos [3].* Exploits commutativity across multiple leaders to achieve lower latency in geo-distributed settings. Unlike CURP, it requires no witnesses but adds complexity to recovery.

*NOPaxos / Speculative Paxos [4].* Achieve near-1-RTT latency using specialized network hardware (SDN switches). CURP achieves similar latency without hardware modifications.

*BFT-Raft [5].* Extends Raft with Byzantine fault tolerance via digital signatures, a complementary dimension to our latency optimization.

## 8 CONCLUSION

We have designed, implemented, and evaluated a CURP witness layer integrated with Raft for a 5-node replicated key-value store. For non-conflicting workloads the system eliminates one RTT from the critical write path, roughly halving median write latency while preserving Raft's strong consistency guarantees. Performance gracefully degrades toward the Raft baseline as the conflict rate or key skew increases. Correctness is confirmed by an extended

test suite including fault injection under leader crashes, follower failures, and concurrent conflicting writes.

Future work includes: (1) exploiting read commutativity to further widen the fast path, (2) adaptive GC batching to reduce conflicttable pressure at high throughput, and (3) evaluating the system on a multi-machine cluster to measure true network RTT improvements rather than localhost proxies.

## REFERENCES


[1] D. Ongaro and J. Ousterhout. In search of an understandable consensus algorithm. In *USENIX Annual Technical Conference (ATC)*, pages 305–319, 2014.

[2] S. J. Park and J. Ousterhout. Exploiting commutativity for practical fast replication. In *USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2019.

[3] I. Moraru, D. G. Andersen, and M. Kaminsky. There is more consensus in egalitarian parliaments. In *ACM SOSP*, 2013.

[4] J. Li, E. Michael, N. K. Sharma, A. Szekeres, and D. R. K. Ports. Just say no to Paxos overhead: Replacing consensus with network ordering. In *USENIX OSDI*, 2016.

[5] J. Clow and Z. Jiang. A Byzantine fault tolerant Raft. Stanford CS244b Course Project, 2017.


## 9 APPENDIX

The primary AI tool used throughout this project was Claude (claude.ai), specifically Claude Sonnet and Gemini. It was used across multiple phases: understanding the existing codebase, designing the CURP extension architecture, debugging concurrency issues, and writing the benchmark infrastructure.

I

CClaude was most effective as a codebase navigator and explainer. The project involved a large amount of existing infrastructure, Raft consensus, gRPC service definitions, a KV server, test harness, and multiple binaries, that needed to be understood before any CURP-specific work could begin. Feeding the relevant files into context and asking targeted questions about request lifecycles, port layout, and infrastructure patterns saved significant time compared to reading every file cold. The explanations of `RaftTestCtrl`, `ConfigGen`, and the three-port-per-node architecture were accurate and directly usable.

Claude was also effective for debugging specific concurrency bugs. When `put_curp()` crashed with `std::future_error: No associated state`, describing the structure of the retry loop and asking for an explanation immediately surfaced the root cause — calling `.get()` on a consumed future inside a loop — along with a corrected pattern that preserved the intended parallelism. Similarly, the explanation of why background-threaded `propose()` calls cause correctness failures under leader failover and concurrent clients was precise and identified failure modes that would have taken much longer to find through testing alone.

For the benchmark design, Claude helped reason through what latency numbers to expect under correct CURP behavior versus pathological cases, which was useful for interpreting results and distinguishing implementation bugs from expected behavior.

Prompts Used: I have a distributed KV store built on top of Raft consensus. I have kv_client.hpp, kv_server.hpp, and raft.cpp. Can you explain how the existing put(), get(), and append() functions work end to end from client to server to Raft and back? I want to understand the full request lifecycle before adding anything new.

### 9.1 Claude responded: Here's the full lifecycle for each operation:

Here's the full lifecycle for each operation:

Put (and Append — identical flow)

(1) KvClient::put() increments seq_num_, serializes a PutRequest with the RIFL fields (client_id, seq_num), and sends it to whichever stub is currently leader_idx_. If the response is KV_NOTLEADER or the gRPC call fails, it rotates to the next stub and retries up to 50 times.
(2) On the server, KvServer::Put() first checks raft_.get_state().is_leader — if not leader, returns KV_NOTLEADER immediately. Then it checks rifl_cache_ for a duplicate (same client_id + seq_num), returning the cached result if found. Otherwise it serializes the op as "PUT\t{key}{value}{client_id}{seq_num}" and calls raft_.propose(op).
(3) Raft::propose() acquires the mutex, appends a new raftpb::Entry to log_entries, updates match_index and next_index for itself, then releases the lock and calls signal_replication() to wake the per-follower replication workers.
(4) Each replication_worker calls replicate_to_follower(), which sends an AppendEntries RPC carrying the new entry. On success, the leader advances match_index for that follower and checks whether a majority now has the entry — if so, advances commit_index and calls apply() for each newly committed entry.
(5) apply() enqueues an ApplyResult into ready_queue. In kv_node.cpp, KvNodeWrapper::Apply() dequeues from that queue and calls kv_server_->on_apply(result).
(6) on_apply() deserializes the op string, checks rifl_cache_ again for duplicates, mutates store_ (for PUT: store_[key] = value), updates rifl_cache_, then fulfills the std::promise in waiters_[result.index] — which unblocks the waiting RPC handler thread.
(7) The RPC handler thread was blocked on fut.wait_for(5s) after calling register_waiter(proposal.index). Once the promise is fulfilled it gets the OpResult, sets the response status, and returns to the client.

Get

The flow diverges in two places. First, before proposing anything, the server does a stale read check: if has_valid_lease() is false (no recent quorum ACK within 100ms), it calls read_quorum_barrier(), which sends AppendEntries to a quorum of followers to confirm it's still leader. If that fails, it returns KV_NOTLEADER. Second, instead of proposing to Raft, it reads commit_index directly and then waits (via kv_applied_cv_) until last_kv_applied_up_to_ >= commit_index — meaning all committed entries have been applied to store_. Then it reads directly from store_ without going through the log at all.

Where the current code actually is

Worth noting: the Put and Append handlers in the current kv_server.hpp have their normal flow commented out. They're currently doing the fast-path CURP version — returning KV_SUCCESS immediately and enqueuing the proposal to a background thread — rather than the standard wait-for-commit flow. The commented-out block is the correct standard path. The Get handler is using the standard path.

## 9.2 You said: Now explain CURP to me.

Now explain CURP to me. What is a witness? What is a superquorum? What is the fast path vs slow path? What is speculative execution? Use the existing put() flow as a reference point to contrast with how put_curp() would work differently.

## 9.3 Claude responded: The core problem CURP solves

The core problem CURP solves

Normal Raft Put takes 2 RTTs minimum: client → leader (RTT 1), leader replicates to followers and commits (RTT 2), then responds. CURP's insight is that if an operation doesn't conflict with anything in flight, you can safely acknowledge it to the client after just 1 RTT, before it's durably in the Raft log.

Witnesses

Every node (not just the leader) acts as a witness. A witness maintains an in-memory table of "unsynced ops" — operations the client has sent that haven't yet been committed to the Raft log. When a client multicasts an op to all nodes, each node records it in its witness table via witness_record(). The key thing a witness checks is commutativity: does this incoming op conflict with anything already in its table? For a KV store, conflict means two ops touching the same key where at least one is a write. If no conflict, the witness records it and ACKs. If conflict, it denies the fast path.

In your raft.cpp, witness_record() does exactly this — it iterates unsynced_ops_ and checks if any existing op has the same key as the incoming write. If yes, returns conflict=true. If no, inserts into unsynced_ops_ with a new witness_idx and returns conflict=false.

Superquorum

Normal Raft only needs a majority quorum (f+1 of 2f+1 nodes) to commit. CURP requires a superquorum for the fast path: f + f/2 + 1 nodes. For a 5-node cluster with f=2, that's 4 out of 5. The reason is recovery safety — if the leader crashes after fast-path ACKing but before Raft commits, a new leader must be guaranteed to find the op on enough witnesses to recover it. With a superquorum of witnesses plus a normal Raft majority for the new leader election, their intersection is always non-empty, so the op is never lost.

In kv_client.hpp, witness_superquorum_record() fires async RPCs to all stubs simultaneously and counts conflict-free ACKs. It returns true only when required_ok (4 for a 5-node cluster) are collected without any conflict.

Fast path vs slow path

Fast path: client multicasts to all nodes simultaneously. If a superquorum of witnesses all ACK with no conflict, the client considers the op committed and returns immediately — without waiting for Raft to log it. The leader still asynchronously proposes the op to Raft in the background (the GC path), but the client doesn't wait for that.

Slow path: if any witness reports a conflict, or the superquorum isn't reached in time, the client falls back to the normal Raft path — send to leader, wait for majority replication, receive commit confirmation. This is exactly what the original put() does.

Speculative execution

"Speculative" means the client's fast-path ACK is a guarantee, not a guess — but only because the system is designed to make it true. Since a superquorum of witnesses recorded the op, any future leader elected after a crash will contact a quorum of witnesses via GetRecoveryData, find the op, and re-propose it to Raft before serving new requests. So the op will definitely end up committed. The client's success response is "speculative" in the sense that Raft hasn't logged it yet, but it's durable in the sense that the witness records ensure it will be.

put() vs put_curp() contrasted

In normal put(), the client sends to one node (the leader), waits for the full Raft commit cycle, and gets a response. Two sequential RTTs minimum.

In put_curp(), the client does two things in parallel: it fires witness_superquorum_record() as an async future, and simultaneously sends the standard Put RPC to the leader. Then it waits for both to complete. There are three outcomes:

- Leader RPC succeeds and witnesses succeeded → fast path, return immediately. The op is guaranteed durable via witnesses even if the Raft log hasn't caught up.
- Leader RPC succeeds but witnesses failed (conflict or insufficient ACKs) → the op went through Raft normally, but you can't trust the fast-path guarantee. The code calls sync_latest() to confirm the current committed state is visible before returning.
- Leader RPC fails → rotate leader, retry.

The key difference from normal put() is that the leader RPC and the witness multicast happen concurrently. In the non-conflicting case, the leader can return as soon as it receives the op (before Raft commits it) because it knows the witnesses have it covered. In your current kv_server.hpp, the Put handler actually does this — it returns KV_SUCCESS immediately and enqueues the proposal to a background thread, trusting that witnesses ensure durability.

## 9.4 You said: I need to write a 5-node cluster harness that wraps the existing RaftTestCtrl infrastructure from Lab 3.

I need to write a 5-node cluster harness that wraps the existing RaftTestCtrl infrastructure from Lab 3. Looking at how multinode.cpp and the existing latency.cpp set up a 3-node cluster, can you explain what steps I need to replicate for 5 nodes? What is ConfigGen::gen_local_instances doing? What is RaftTestCtrl? What ports need to be configured? Give me pseudocode for a CurpCluster class with start(), stop(), kill_node(), revive_node(), and make_client() methods.

## 9.5 Claude responded: Looking at multinode.

Looking at multinode.cpp and the lab 3 benchmark infrastructure, here's what each piece does and then the pseudocode.

ConfigGen::gen_local_instances

Takes a count and a base port, returns a vector of instance descriptors. Each descriptor has an id (0-indexed), a listening_addr (0.0.0.0:{port+i}), and an external_addr (localhost:{port+i}). The distinction matters: listening_addr is what the gRPC server binds to, external_addr is what peers use to connect. For 5 nodes starting at 50050, you get ports 50050–50054 for Raft, which means KV ports are 51050–51054 (the KV_PORT_OFFSET = 1000 from kv_node.cpp).

RaftTestCtrl

This is the cluster manager. It spawns one kv_node subprocess per config, wires up tester gRPC ports for control (kill, reconnect, propose), and provides run(), kill(), disconnect(ids), reconnect(ids), and propose_to_all(data). The tester ports are separate from both Raft and KV ports — they start at 55001 by convention and increment per node. Each kv_node process exposes all three: Raft port P, KV port P+1000, tester port T.

What you need to replicate for 5 nodes

From multinode.cpp the steps are: generate instances → build per-node rafty::Config with peer map (everyone except self) → assign tester ports → build logger → construct RaftTestCtrl → call ctrl->run(). For KvClient you separately build the KV address list from the external addrs + 1000 offset.

Pseudocode

```
class CurpCluster {
// owns the cluster lifecycle
unique_ptr<RaftTestCtrl> ctrl;
vector<string> kv_addrs;      // "localhost:5105X" for each node
int num_nodes;
int base_port;
string ctrl_addr;
shared_ptr<spdlog::logger> logger;
// track which nodes are currently alive for revive
map<uint64_t, rafty::Config> all_configs;
map<uint64_t, uint64_t> all_tester_ports;
void start(int n = 5, int base = 50050) {
num_nodes = n;
base_port = base;
ctrl_addr = "0.0.0.0:55000";
// 1. generate instance descriptors
auto insts = ConfigGen::gen_local_instances(n, base_port);
// 2. build rafty::Config for each node
//    peer_addrs = everyone else's external_addr
vector<rafty::Config> configs;
map<uint64_t, uint64_t> tester_ports;
uint64_t tester_port = 55001;
for each inst in insts:
map<uint64_t, string> peer_addrs;
for each other in insts where other.id != inst.id:
peer_addrs[other.id] = other.external_addr;  // "localhost:5005X"
configs.push_back({
.id = inst.id,
.addr = inst.listening_addr,   // "0.0.0.0:5005X"
.peer_addrs = peer_addrs
});
tester_ports[inst.id] = tester_port++;
all_configs[inst.id] = configs.back();
all_tester_ports[inst.id] = tester_ports[inst.id];
// 3. KV addresses (Raft port + 1000)
for each inst in insts:
kv_addrs.push_back("localhost:" + to_string(inst.external_port + 1000));
// 4. logger
logger = spdlog::basic_logger_mt("curp_cluster", "logs/curp_cluster.log");
// 5. construct and start controller
//    binary path is "./kv_node", fail_type=0, verbosity=1
ctrl = make_unique<RaftTestCtrl>(
configs, tester_ports, "./kv_node",
ctrl_addr, /*fail_type=*/0, /*verbosity=*/1, logger
);
ctrl->run();
// 6. give cluster time to elect a leader
sleep(2s);
}
void stop() {
if ctrl: ctrl->kill();
ctrl.reset();
}
// disconnect node id from the rest of the cluster
// RaftTestCtrl::disconnect takes a vector<uint64_t>
void kill_node(uint64_t id) {
ctrl->disconnect({id});
}
// reconnect a previously disconnected node
void revive_node(uint64_t id) {
ctrl->reconnect({id});
}
// returns a fresh KvClient pointed at all KV ports
// caller decides whether to use put() or put_curp()
kv::KvClient make_client() {
return kv::KvClient(kv_addrs);
}
// convenience: make a client that only sees a subset of nodes
// useful for testing partition behavior
kv::KvClient make_client(vector<uint64_t> node_ids) {
vector<string> addrs;
for id in node_ids:
addrs.push_back(kv_addrs[id]);
return kv::KvClient(addrs);
}
}
```

Port layout for 5 nodes

```
node 0: Raft=50050  KV=51050  tester=55001
node 1: Raft=50051  KV=51051  tester=55002
node 2: Raft=50052  KV=51052  tester=55003
node 3: Raft=50053  KV=51053  tester=55004
node 4: Raft=50054  KV=51054  tester=55005
```

One thing to watch: RaftTestCtrl uses ctrl_addr (0.0.0.0:55000) as the controller's own listener, separate from the per-node tester ports. The node tester ports (55001–55005) are what each kv_node subprocess listens on for tester control RPCs. Make sure 55000 isn't in the tester port range.

## 9.6 You said: I want to track what percentage of putcurp() calls took the 1-RTT fast path vs fell back to the slow path.

I want to track what percentage of put_curp() calls took the 1-RTT fast path vs fell back to the slow path. Looking at put_curp() in kv_client.hpp, where exactly in the code is it determined whether the fast path succeeded or not? How would I add counters to track this without adding extra RPCs or polluting the witness state? Give

me pseudocode for where to increment the counters and what methods to expose.

## 9.7 Claude responded: Where the fast/slow path decision is made in put_curp()

Where the fast/slow path decision is made in put_curp()

The decision point is right after both futures resolve — the leader RPC result and witness_fut.get(). The logic is:

- If the leader RPC succeeded AND witnesses_ok is true → fast path taken
- If the leader RPC succeeded AND witnesses_ok is false → slow path (fell back to sync_latest())
- If the leader RPC failed → neither path succeeded, retry

So the branch you care about is entirely local to the client, after both results are in hand. No extra RPCs needed — the information is already there.

Where to put the counters

Two atomic counters as members of KvClient: one for fast path successes, one for slow path fallbacks. Increment fast path in the branch where both the leader RPC succeeded and witnesses returned true. Increment slow path in the branch where the leader RPC succeeded but witnesses returned false, triggering sync_latest(). Don't increment anything on retry/failure — those aren't path decisions, they're retries.

Pseudocode

```
class KvClient {
// existing members...
atomic<uint64_t> fast_path_count;
atomic<uint64_t> slow_path_count;
put_curp(key, value):
fire witness_superquorum_record asynchronously
loop up to max_attempts:
send Put RPC to current leader
collect witness result
if leader RPC succeeded:
if witnesses_ok:
increment fast_path_count   // <– fast path
return SUCCESS
else:
increment slow_path_count   // <– slow path
sync_latest()
return SUCCESS
rotate leader, sleep, retry
return TIMEOUT
// expose counters
uint64_t get_fast_path_count()
uint64_t get_slow_path_count()
double get_fast_path_ratio():
total = fast_path_count + slow_path_count
if total == 0: return 0.0
return fast_path_count / total
void reset_counters():
fast_path_count = 0
slow_path_count = 0
}
```

One subtlety to be aware of

put_curp() fires the witness future and the leader RPC concurrently, but then calls witness_fut.get() inside the retry loop after the leader RPC returns. If the leader RPC fails and you rotate and retry, you've already consumed the witness future — it can't be re-awaited. So in practice the counter increment only happens on the first attempt where the leader RPC succeeds. That's the right behavior: you only count a path decision when an attempt actually completes, not on transient failures mid-retry.

## 9.8 You said: I need to write curplatency.

I need to write curp_latency.cpp that measures latency of put() vs put_curp() and reports avg, p50, p99, and fast-path hit rate. Looking at the existing latency.cpp as a reference, what changes do I need to make to support 5 nodes instead of 3, support a –curp flag to switch between put() and put_curp(), and report fast-path hit rate? Give me pseudocode for the main() function

## 9.9 Claude responded: What changes from latency.

What changes from latency.cpp

The cluster setup changes from 3 nodes to 5 — everything else in the config generation and controller setup is the same pattern, just a different count passed to gen_local_instances. The KV address list will have 5 entries instead of 3 but the construction logic is identical.

The client needs to be either a plain KvClient (for put()) or a modified one with counters (for put_curp()). Since both live in the same class, the switch is just which method you call per operation, gated on the flag.

The output section gains one extra row or column for fast-path hit rate, which is only meaningful when –curp is set.

Pseudocode for main()

```
parse flags:
–curp (bool, default false)
–ops  (int, default 1000)
–key  (string, default "bench_key")
setup cluster with 5 nodes (same pattern as latency.cpp but n=5)
wait for leader election
create KvClient pointed at all 5 KV addresses
warmup: send a few operations to ensure leader is discovered
latencies = empty list
for i in 0..num_ops:
record start time
if curp_flag:
client.put_curp(key, value)
else:
client.put(key, value)
record end time
append (end - start) to latencies
sort latencies
compute:
avg = sum(latencies) / len(latencies)
p50 = latencies[len * 0.50]
p99 = latencies[len * 0.99]
print latency table (same format as latency.cpp)
if curp_flag:
```

print fast-path hit rate = client.get_fast_path_ratio()
print fast_path_count and slow_path_count raw numbers
teardown cluster
A few things to think through

The –key flag matters more than it looks. If all 1000 ops hit the same key, every CURP op after the first will conflict in the witness table because the previous op may not have been GC'd yet. That means the fast-path hit rate will be artificially low and not reflective of CURP's actual benefit. You probably want to either use random keys from a pool, or use a fixed key but understand you're measuring the conflicting-write case intentionally.

The warmup matters because the first few ops will include leader discovery time (the client rotating until it finds the leader), which skews p99 badly. A handful of throwaway ops before starting the timer fixes this.

For the output, consider printing the mode (plain Raft vs CURP) as a header line so results from different runs are self-labeling when you save them to a file.

## 9.10 You said: My putcurp() is crashing with std::futureerror: No associated state.

My put_curp() is crashing with std::future_error: No associated state. I have a std::future created by std::async and I'm calling .get() on it inside a retry loop. Can you explain why this crashes and what the correct pattern is for using std::future in a retry loop? Give me pseudocode for the corrected put_curp() structure

## 9.11 Claude responded: Why it crashes

Why it crashes

std::future is move-only and can only be .get()'d once. After .get() is called, the future has no associated state — calling .get() again throws std::future_error. In put_curp(), the witness future is created once before the retry loop, but .get() is called inside the loop. If the leader RPC fails and the loop iterates a second time, it tries to .get() an already-consumed future, which is the crash.

The structural problem

The witness future and the retry loop have mismatched lifetimes. The future is a one-shot result, but the loop assumes it can be re-read on every iteration.

The correct pattern

The witness result should be extracted from the future exactly once, outside or at a fixed point in the loop, and stored in a plain variable. The retry loop then reads from that variable, not from the future directly.

```
put_curp(key, value):
increment seq_num
// launch witness multicast once before the loop
witness_future = async(witness_superquorum_record(...))
// consume the future exactly once, before or at start of loop
witnesses_ok = witness_future.get()
// now retry loop operates only on plain values
for attempt in 0..max_attempts:
send Put RPC to current leader
if leader RPC succeeded:
if witnesses_ok:
increment fast_path_count
return SUCCESS
else:
increment slow_path_count
sync_latest()
return SUCCESS
// RPC failed, rotate leader and retry
// witnesses_ok doesn't change — we already have the witness result
rotate_leader()
sleep()
return TIMEOUT
```

The tradeoff this introduces

By calling .get() before the leader RPC loop, you're now waiting for the witness future to complete before sending any retries. In the original code, the intent was to overlap the witness multicast with the first leader RPC attempt. You can preserve that overlap by restructuring slightly — launch the future, attempt the leader RPC once without looping, then call .get() after that first attempt returns. If the first attempt fails, you already have the witness result in hand and the retry loop runs purely on plain values from there.

```
put_curp(key, value):
increment seq_num
witness_future = async(witness_superquorum_record(...))
// first attempt overlaps with witness multicast
send Put RPC to current leader
witnesses_ok = witness_future.get()   // collect after first RPC returns
if first RPC succeeded:
handle success/failure of witnesses_ok as above
return
rotate_leader()
sleep()
// subsequent attempts have no witness future to worry about
for attempt in 1..max_attempts:
send Put RPC to current leader
if leader RPC succeeded:
if witnesses_ok:
increment fast_path_count
return SUCCESS
else:
increment slow_path_count
sync_latest()
return SUCCESS
rotate_leader()
sleep()
return TIMEOUT
```

This preserves the 1-RTT overlap on the happy path while keeping the future consumed exactly once.

## 9.12 You said: My correctness tests are failing with KVTIMEOUT even though the server is responding KVSUCCESS.

My correctness tests are failing with KV_TIMEOUT even though the server is responding KV_SUCCESS. The server is using a background thread to call raft_.propose() after responding to the client. Can you explain why this causes correctness issues and what the

correct server-side behavior should be for both the CURP fast path and the baseline slow path? Give me pseudocode for both implementations

## 9.13 Claude responded: Why the background proposal causes correctness failures

Why the background proposal causes correctness failures

When the server responds KV_SUCCESS before the op is in the Raft log, the client thinks the write is committed. But if the client immediately issues a Get, that Get goes through read_quorum_barrier and waits until last_kv_applied_up_to >= commit_index. The problem is the background-proposed op may not be committed yet — or worse, if the leader steps down between responding and proposing, the op is never proposed at all. The client got a success response for a write that never happened.

The deeper issue is that responding before propose() means you've made a durability promise you haven't kept. RIFL makes this worse — if the client retries and hits a different server, the rifl_cache_ entry doesn't exist there, so the op gets re-executed. But if it hits the same server, the cache has the success response even though the op was never committed, so it returns success for a write that's still missing from the log.

Three specific failure modes in the tests

First, a Get after a Put returns stale data because the background proposal hasn't committed yet when the read fence is evaluated. Second, a leader failover between the response and the background proposal drops the write entirely — the new leader has no record of it. Third, concurrent clients can violate linearizability because the ordering of background proposals is non-deterministic relative to other in-flight ops.

What the correct slow path should do

The server must not respond until the op is durably committed in the Raft log and applied to store_. The flow is synchronous from the server's perspective.

```
Put_slow_path(request):
if not leader:
return KV_NOTLEADER
if rifl_cache has (client_id, seq_num):
return cached result
serialize op string
proposal = raft_.propose(op)
if not proposal.is_leader:
return KV_NOTLEADER
register waiter at proposal.index
wait for waiter to be fulfilled (with timeout):
if timeout:
mark index as abandoned
return KV_TIMEOUT
return result from waiter
```

The waiter is fulfilled by on_apply() when Raft commits and applies the entry at that index. The client gets KV_SUCCESS only after the op is in the log.

What the correct CURP fast path should do

The CURP server-side fast path is more subtle. The leader still needs to call propose() synchronously — it just doesn't need to wait for the commit to respond to the client, because the witness superquorum guarantees durability. But critically, propose() itself must be called before responding, so the op enters the Raft pipeline. The response can come early, but the proposal cannot be deferred.

```
Put_curp_server_side(request):
if not leader:
return KV_NOTLEADER
if rifl_cache has (client_id, seq_num):
return cached result
serialize op string
// propose synchronously so op enters Raft pipeline
proposal = raft_.propose(op)
if not proposal.is_leader:
return KV_NOTLEADER
// record that this index came via fast path
// on_apply() will apply it but not try to fulfill a waiter
mark proposal.index as fast_path_index
// respond immediately — witness superquorum guarantees durability
return KV_SUCCESS
```

The on_apply() handler needs to know about fast path indices so it applies the op to store_ and updates rifl_cache_ without trying to fulfill a promise that doesn't exist.

What on_apply() needs to handle for both cases

```
on_apply(result):
deserialize op
check rifl_cache for duplicate, compute op_result
apply to store_ if not duplicate
update rifl_cache_
if waiters_ has result.index:
// slow path: fulfill the waiting RPC handler
fulfill waiter with op_result
else if fast_path_indices_ has result.index:
// fast path: op already responded to client, just clean up
remove from fast_path_indices_
else:
// RPC handler timed out before apply arrived
if not in abandoned_indices_:
buffer in applied_results_
else:
remove from abandoned_indices_
update last_kv_applied_up_to_
notify kv_applied_cv_
```

The key invariant

In both paths, raft_.propose() is called synchronously before any response leaves the server. The only difference is whether the server waits for on_apply() before responding. Slow path waits. Fast path does not, but relies on the client having already secured a witness superquorum to guarantee the op survives any leader crash between propose and commit.

## 9.14 You said: My latency benchmark shows CURP fast path is slower than baseline Raft.

My latency benchmark shows CURP fast path is slower than baseline Raft. What are the possible reasons for this? How would I diagnose whether the issue is on the client side, server side, or

in the witness recording? What numbers should I expect to see if everything is working correctly and why

## 9.15 Claude responded: Why CURP could be slower than baseline

Why CURP could be slower than baseline

There are several distinct causes, each pointing to a different layer.

Client-side causes

The witness multicast fires stubs_.size() async RPCs simultaneously. If those async launches have high overhead — thread creation cost per std::async call — the multicast itself takes longer than a single leader RPC. std::async with std::launch::async spawns a new thread every call on most implementations. For a 5-node cluster that's 5 threads launched per operation. At 1000 ops that's 5000 thread creations, which dominates latency.

The second client-side issue is that witness_future.get() blocks until the slowest witness responds. With 5 nodes you need 4 ACKs, but you still wait for all 5 futures to resolve in the current implementation because the loop collects all of them sequentially. If one witness is slow, the fast path is gated on it even though you already have your 4 ACKs.

Server-side causes

witness_record() acquires witness_mtx_ which is separate from mtx. But the leader's propose() also runs and acquires mtx. If the witness RPC handler and the propose call are serialized due to lock contention on the gRPC thread pool, you're adding latency on top of the normal Raft path rather than overlapping with it.

The background GC is another issue. If witness_gc() is called too infrequently, unsynced_ops_ grows large and the linear scan in witness_record() over every entry in the map gets slower with each op. For a sequential single-key benchmark this is especially bad — every new op scans all previous unsynced entries.

Network/RPC causes

In a local cluster all nodes are on localhost, so actual network latency is near zero. But gRPC has per-RPC overhead from serialization, context creation, and HTTP/2 framing. The baseline put() does one RPC. put_curp() does one leader RPC plus up to 5 witness RPCs concurrently. Even if the witness RPCs are parallel, you're doing more total work, and the gRPC overhead adds up on localhost where compute cost dominates over wire latency.

How to diagnose each layer

To isolate the client-side thread overhead, time just the witness_superquorum_record() call in isolation without any leader RPC. If that alone takes longer than a full baseline put(), the async thread launch cost is your bottleneck.

To isolate the witness scan cost, run the benchmark with a large key pool so each op touches a different key. If latency is flat regardless of how many ops you've sent, the scan is not the issue. If latency grows as ops accumulate, GC is falling behind.

To isolate the slow-witness problem, add per-witness timing inside witness_superquorum_record() and log how long each future takes. If one node consistently takes much longer than the others, you have a stragggler problem and should return as soon as you hit the required count rather than waiting for all futures.

To isolate server-side lock contention, add timing around witness_record() on the server and compare it to the time propose() takes. If both are fast individually but together are slow, they're serializing somewhere.

What numbers to expect if everything is working correctly

On a localhost 5-node cluster with no artificial delays, baseline put() should be in the range of 5-15ms median. This reflects one client-to-leader RTT plus Raft replication to a majority of followers plus the commit and apply path.

CURP put_curp() on a non-conflicting workload should be faster than baseline, not slower. The theoretical gain is that the client gets its response as soon as the leader receives the op and the witness superquorum ACKs, without waiting for Raft commit. On localhost that should be closer to 2-5ms median — roughly the cost of one RPC plus the parallel witness multicast, where the witness multicast overlaps with the leader RPC.

If your p50 for CURP is higher than baseline p50, the overhead of the witness multicast is exceeding the savings from skipping the Raft commit wait. The most likely culprit is thread creation cost. If your p99 for CURP is much higher than baseline p99, the stragglerwitness problem is dominating tail latency.

Fast-path hit rate on a single-key sequential workload should be near zero, because each op conflicts with the previous unsynced op on the same key. On a large random key pool with no hot keys it should be near 100%. If you're seeing a hit rate below 90% on a non-conflicting workload, the GC is not keeping up and stale witness entries are causing false conflicts.